

ACVP X-WING JSON Specification

Abstract

This document defines the JSON schema for testing General-purpose hybrid post-quantum key encapsulation mechanism (KEM) specification combining the classical X25519 elliptic curve algorithm and the post-quantum ML-KEM-768 algorithm implementations with the ACVP specification.

Introduction

The Automated Crypto Validation Protocol (ACVP) defines a mechanism to automatically verify the cryptographic implementation of a software or hardware crypto module. The ACVP specification defines how a crypto module communicates with an ACVP server, including crypto capabilities negotiation, session management, authentication, vector processing and more. The ACVP specification does not define algorithm specific JSON constructs for performing the crypto validation. A series of ACVP sub-specifications define the constructs for testing individual crypto algorithms. Each sub-specification addresses a specific class of crypto algorithms. This sub-specification defines the JSON constructs for testing General-purpose hybrid post-quantum key encapsulation mechanism (KEM) specification combining the classical X25519 elliptic curve algorithm and the post-quantum ML-KEM-768 algorithm implementations using ACVP.

This document is a work-product of the [NIST Cryptographic Algorithm Validation Program](#) (CAVP). Please communicate questions and feedback concerning this document to the CAVP by opening an issue on the [usnistgov/ACVP](https://github.com/usnistgov/ACVP) GitHub repository at <https://github.com/usnistgov/ACVP/issues>.
include::common/common-sections/02-conventions.adoc[

Supported X-WING Algorithms

The following X-WING algorithms **MAY** be advertised by the ACVP compliant cryptographic module. The list is in the form "algorithm / mode / revision".

- X-WING / keyGen / v1_0
- X-WING / encapDecap / v1_0

Test Types and Test Coverage

Test Types

The ACVP server performs a set of tests on the specified X-WING algorithm in order to access the correctness and robustness of the implementation. A typical ACVP validation session **SHALL** require multiple tests to be performed for every supported permutation of X-WING capabilities. This section describes the design of the tests used to validate implementations of the X-WING

algorithms.

The IUT REQUIRED for each test case provided to generate an X-WING key pair from the provided 32-byte seed. The generated encapsulation key (pk) and decapsulation key (sk) are communicated to the ACVP server and validated. This tests the implementation of the X-WING GenerateKeyPair() procedure and its deterministic variant GenerateKeyPairDerand(), as defined in Section 5.2 and Section 5.2.1 of the X-WING specification.

For encapsulation testing, the server will generate a deterministic encapsulation seed ('eseed') and an encapsulation public key ('pk'). The implementation under test (IUT) shall perform the X-WING encapsulation operation and return the ciphertext ('ct') and shared secret ('ss').

X-WING / encapDecap / "VAL"

Validation Test (VAL).

For decapsulation validation, the server shall generate a valid X-WING decapsulation key ('dk') together with a corresponding valid ciphertext ('ct'). The IUT shall perform the X-WING Decapsulate() operation and return the resulting shared secret ('ss').

Validation testing includes both successful decapsulation and modified-ciphertext test cases. For modified-ciphertext test cases, the server shall independently modify the ML-kem ciphertext component ('ct_m') and the X25519 ciphertext ('ct = ct_M || ct_X'). The IUT shall correctly process each modified ciphertext and return the resulting shared secret ('ss').

These tests verify the implementation of the X-WING decapsulation procedure defined in Section 5.5 of [\[repo:\(IETF D.connolly-cfrg-xwing-kem\)\]](#)

Required Prerequisite Algorithms for X-WING Validations

The implementation relies on two underlying core algorithms:

- MK_KEM-768: Conforming to FIPS 203 (Module-Lattice-Based Key Encapsulation mechanism)
- X25519: Conforming to SP 800-186 / RFC7748 for elliptic-curve cryptography.[1, 2]

Prerequisite Validations

Before XWING algorithm validation can be executed, the underlying cryptographic libraries must demonstrate successful validation of the following prerequisite algorithm primitives:

Table 1. Prerequisite algorithm primitives

Prerequisite Primitive	Standard Reference	Operational Purpose
SHA3-256 / SHAKE256	FIPS202	Message hashing and internal domain separation matrix generation within ML-KEM.

KDF (HKDF-SHA256)	SP 800-56C	Hybrid key derivation and combination layer for component secrets.
-------------------	------------	--

ACVP Validation Interface & JSON Profile

The validation process uses standard ACVP JSON profiles to communicate registration data, test vectors, and response structures.

Algorithm Registration Capabilities

The following JSON payload outlines the capabilities registration block sent by the client to request XWING test vector challenges:

Registration Block Schema

```
{
  "acvVersion": "1.1.0",
  "algorithms": [
    {
      "id": 0,
      "algorithm": "XWING",
      "revision": "1.0",
      "capabilities": [
        {
          "mode": "keyGen",
          "role": ["initiator", "responder"]
        },
        {
          "mode": "encapsulation",
          "role": ["initiator"]
        },
        {
          "mode": "decapsulation",
          "role": ["responder"]
        }
      ]
    }
  ]
}
```

X-WING encapDecap Mode Capabilities Example

Below is an example of the registration for X-WING / encapDecap / v1_0

encapDecap Mode Capabilities Schema

```
----- {
```

```

    "algorithm": "X-WING",
    "mode": "encapDecap",
    "revision": "1.0",
    "prereqVals": [
        {
            "algorithm": "SHA",
            "valValue": "123456"
        }
    ],
    "parameterSet": [ "ML-KEM-768" ],
    "function": [ "encapsulation", "decapsulation" ]
}
-----

```

X-WING keyGen Mode Capabilities Example

Below is an example of the registration for X-WING / keyGen / v1_0

keyGen Registration Schema

```

{
    "algorithm": "X-WING",
    "mode": "keyGen",
    "revision": "1.0",
    "prereqVals": [
        {
            "algorithm": "SHA",
            "valValue": "123456"
        }
    ],
    "function": [
        "encapsulation",
        "decapsulation"
    ]
}

```

X-WING encapDecap Test Vectors

X-WING encapDecap Test Groups JSON Schema

The examples in this section illustrate the JSON exchanges between the ACVP server and the implementation under test (IUT) for X-WING encapsulation and decapsulation testing.

Encapsulation Schema

```

{
    "tgId": 1,
    "tests": [
        {

```

```

    "tcId": 1,
    "pk": "A1B2C3",
    "eseed": "D2E3F4"
  }
]
}

```

Table 2. XWING encapDecap Test Case JSON Object

Property	Description
pk	X-WING public (encapsulation) key
eseed	Deterministic encapsulation seed
tcId	Test case Identifier

Decapsulation Schema

```

{
  "tgId": 2,
  "tests": [
    {
      "tcId": 26,
      "ss": "1D1AE"
    }
  ]
}

```

Table 3. Prompt JSON (Decapsulation)

Property	Description
dk	X-WING decapsulation key.
ct	X-WING ciphertext.
tcId	Test case Identifier.

Validation

Validation test vectors exercise both valid and modified ciphertexts. The server independently modifies the ML-Kem ciphertext component (ct_M) and the X25519 ciphertext component (ct_X) to verify that the implementation correctly processes each portion of the combined X-WING ciphertext.

Test Vector Validation

```

{
  "tgId": 3,
  "tests": [
    {
      "tcId": 29,

```

```

    "ss": "...",
    "reason": "modified ctM"
  },
  {
    "tcId": 30,
    "ss": "...",
    "reason": "modified ctX"
  }
]
}

```

Table 4. Test Case Validation Table

Property	Description
ss	Shared secret returned by decapsulation.
reason	Validation disposition (modified ctM or modified ctX)
tcId	Test case Identifier.

X-WING Key Generation Test Vectors

X-WING Key Generation Test Groups JSON Schema Intro

The Key Generation test vectors verify the implementation correctly performs the X-WING GenerateKeyPair() operation. The server supplies the deterministic 32-byte SkSeed (seed). The implementation under test (IUT) shall generate the corresponding X-WING public (encapsulation) key 9pk) and decapsulation key (dk).

These tests verify the implementation of the x-WING GenerateKeyPair() procedure defined on Section 5.2 of [\[repo:\(IETF D.connolly-cfrg-xwing-kem\)\]](#).

Prompt JSON

```

{
  "tgId": 1,
  "tests": [
    {
      "tcId": 1,
      "seed": "0123456789ABCDEF..."
    }
  ]
}

```

Table 5. Generate X-WING Key Pair

Property	Description
----------	-------------

seed	Deterministic seed used to generate the X-WING key pair.
tcId	Test case Identifier.

KeyGen Test Case

```
{
  "tgId": 1,
  "tests": [
    {
      "tcId": 1,
      "pk": "A1B2C3D4...",
      "dk": "E5F60718..."
    }
  ]
}
```

Table 6. X-WING keyGen Test Case JSON Object

Property	Description
pk	X-WING public (encapsulation) key.
dk	X-WING decapsulation key.
tcId	test case identifier.

Test Vectors

The X-WING ACVP server provides test vectors as JSON objects. Each test vector set consists of one or more test groups, with each group containing one or more test cases. The top-level JSON object contains metadata identifying the algorithm, mode of operation, revision, and associated test groups.

Table 7. Top Level Test Vector JSON Elements

Property	Description	JSON Type
acvVersion	Version of the ACVP protocol specification.	string
vsId	Vector set identifier assigned by the ACVP server.	integer
algorithm	Algorithm name. The value shall be "X-WING".	string
mode	Mode of operation supported by the vector set (for example, keygen or encapDecap)	string

Property	Description	JSON Type
revision	X-WING specification revision supported by the implementation.	string
testGroups	Array of one or more test groups.	array

Example Test Vector Element

```
{
  "acvVersion": "1.0",
  "vsId": 42,
  "algorithm": "X-WING",
  "mode": "encapDecap",
  "revision": "1.0",
  "testGroups": [
    {
    }
  ]
}
```

X-WING encapDecap Test Vector Responses

The X-WING EncapDecap responses object contains the results generated by the implementation under test (IUT) after processing the supplied test vectors. Response objects contain the values computed by the IUT and are returned to the ACVP server for validation.

Table 8. Top-Level Response Object

Property	Description
acvVersion	Version of the ACVP protocol
vsId	Vector set identifier assigned by the ACVP server.
testGroups	Array of X-wing test Groups.

Table 9. X-WING encapDecap Test Groups Response JSON Object

Property	Description
tgId	Test group identifier.
tests	Array of response test cases.

Encapsulation Response

For encapsulation, the IUT shall return the generated ciphertext (ct) and shared secret (ss).

Table 10. Property Table

Property	Description
tcId	Test case identifier.
ct	X-WING ciphertext generated by the encapsulation operation.

Encapsulation response Example

```
{
  "tcId": 17,
  "ct": "F4AE2B7C...",
  "ss": "C57A25..."
}
```

Decapsulation Response

For decapsulation, the IUT shall return the shared secret (ss) derived from the supplied decapsulation key 9dk0 and ciphertext (ct).

Table 11. Decapsulation Property Table

Property	Description
tcId	Test case identifier
ss	Shared secret generated by the decapsulation operation.

Decapsulation response Example

```
{
  "tcId": 26,
  "ss": "1D1AE..."
}
```

Validation Response

Validation response contains the shared secret (ss) produced after processing both valid and modified ciphertexts. Modified ciphertext test cases independently exercise the ML-KEM (ct_M) and X25519 (ct_X) components of the combined X-WING ciphertext.

Validation Response Example

```
{
  "acVersion": "1.0",
  "vsId": "42",
  "testGroups": [
    {
```

```

    "tgId": 2,
    "tests": [
      {
        "tcId": 26,
        "ss": "1D1AE..."
      }
    ]
  }
]
}

```

X-WING Key Generation Responses

The X-Wing Key Generation response object contains the key pair generated by the implementation under test (IUT). The response is returned to the ACVP server for validation following successful execution of the X-WING GenerateKeyPair() operation.

Table 12. X-WING Top-Level Response JSON Object

Property	Description
acvVersion	Version of the ACVP protocol.
vsId	Vector set identifier assigned by the ACVP server.
testGroups	Array of X-WING test groups.

Table 13. Test Groups

Property	Description
tgId	Test group identifier.
tests	Array of response test cases.

Test Cases

For each key generation test case, the IUT shall generate the corresponding X-WING public (encapsulation) key (pk) and decapsulation key (dk) from the supplied deterministic seed.

Table 14. Property Table

Property	Description
tcId	Test case identifier.
pk	X-WING public (encapsulation) key generated by the 'GenerateKeyPair()' operation.
dk	X-WING decapsulation key generated by the 'generateKeyPair()' operation.

```
[
  {
    "acvVersion': <acvp-version>
  },
  {
    "vsId": 0,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "pk": "A1B2C3D4",
            "dk": "E1F2G3h4"
          }
        ]
      }
    ]
  }
]
```

X-WING Test Vector Responses

The X-WING response object defines the JSON format returned by the implementation under test (IUT) after processing ACVP test vectors. Response objects contain the values generated by the IUT and are returned to the ACVP server for validation.

Response JSON Structure

The top-level response object contains the following properties.

Table 15. Response JSON Object

Property	Description
acvVersion	Version of the ACVP protocol.
vsId	Vector set identifier assigned by the ACVP server.
testGroups	Array of one or more response test groups.

Response Test Groups

Each response test group contains the results for one or more test cases.

Table 16. Response Test Group Table

Property	Description
tgId	Test group identifier.
tests	Array of response test cases.

Response Test Cases

Each response test case contains the results generated by the IUT for the requested X-WING function.

Table 17. Response Test Case Table

Property	Description
tcId	Test case identifier.
...	Function-specific response values as defined in the corresponding KeyGeneration and EncapDecap response specifications.

Response JSON Test Group Example

```
{
  "arcVersion": "1.0",
  "vsId": 42,
  "testGroups": [
    {
      "tgId": 1,
      "tests": [
        {
          "tcId": 1
        }
      ]
    }
  ]
}
```

Normative References

- [\[repo:\(IETF D.connolly-cfrg-xwing-kem\)\]](#), IETF Internet-Draft: X-Wing Hybrid Key Encapsulation Mechanism
- [RFC 9180]
- [IETF(RFC 7748)]
- [NIST FIPS 203]

Automatic Cryptographic Validation Protocol

id

ACVP

docid

id

ACVP

contributor

contributor.person.name.initial

B.

contributor.person.name.surname

Fussell

contributor.person.affiliation.organization.name

Cisco

contributor

contributor.person.name.initial

A.

contributor.person.name.surname

Vassilev

contributor.person.affiliation.organization.name

National Institute of Standards and Technology

contributor.person.affiliation.organization.abbreviation

NIST

contributor

contributor.person.name.initial

H.

contributor.person.name.surname

Booth

contributor.person.affiliation.organization.name

National Institute of Standards and Technology

contributor.person.affiliation.organization.abbreviation

NIST

contributor

contributor.role

publisher

contributor.organization.name

National Institute of Standards and Technology

contributor.organization.abbreviation

NIST

date

date.type

published

date.value

2019-07-01